

Introduction

Every developer has had this moment: you touch one file, run the app, and something completely unrelated breaks. You didn't touch that file. You didn't even know it existed until it started throwing errors. Your first instinct is to assume you made a mistake somewhere — but when you go looking, the code you touched is clean, and the code that broke is untouched.

This isn't a rare glitch. It's one of the most common early-warning signs that a codebase has no architecture — not "bad" architecture, no architecture at all. And it matters far beyond this one bug, because it's exactly the kind of failure that interviewers probe for in system design rounds: not "can you draw boxes and arrows," but "do you understand why boxes and arrows need to exist in the first place."

Understanding this scenario deeply gives you something most tutorials skip: an intuition for *when* structure earns its keep and when it's just overhead. That distinction is what separates engineers who over-architect three-file scripts from ones who can't scale past ten files, from engineers who know exactly when to add the wall.

Problem Overview

Here's the setup, in plain terms. You have a small application — three files, one folder. Nothing exotic. You add a button to the settings screen. Nothing fancy, just a button.

Now the checkout screen breaks. Not the settings screen — checkout. A screen nobody touched, in a file nobody edited.

The question isn't "how do I fix this bug." The question is: how is this even possible? What has to be true about a codebase for a change in file A to break file B, when file B has zero relationship to what you just did — at least on paper?

That's the real problem this lesson is solving: not a specific bug, but the *structural condition* that makes bugs like this possible at all.

Example

Let's make this concrete instead of abstract.

Imagine `settings.js`, `checkout.js`, and a shared `app.js` that wires them together. You add a button to `settings.js` that reads some shared state to decide whether to render. That state object is the same one `checkout.js` reads to calculate totals. You didn't rename it, you didn't delete it — you just added a new read to it earlier in the app's lifecycle, which changed *when* a value gets set relative to when checkout reads it.

Checkout now sees `undefined` where it expected a number. It breaks.

Notice what didn't happen: nobody edited `checkout.js`. Git blame on that file shows nothing recent. The code inside it is fine, in isolation. The bug isn't a bug in either file — it's a bug in the *relationship* between them, and that relationship was never made explicit anywhere. It just existed, silently, because nothing stopped it from existing.

Intuition

Here's how an experienced engineer reasons through this, step by step, before writing a single line of a fix.

Step one: rule out the obvious. Something broke inside `checkout.js` — did the checkout code itself change? Check git blame. If it's untouched for weeks, that's not it.

Step two: rule out sloppy code. Did whoever wrote `settings.js` do something careless — a typo, a bad mutation, a leaked side effect? Read it. If it's clean, that's not it either.

Step three — the important one: stop looking at the files, and start looking at the connections between the files. If neither file is individually broken, but the system as a whole is broken, the bug isn't inside a file. It's between files. And a bug that lives "between" files can only exist if those files are allowed to reach into each other freely, with no boundary stopping them.

This is the core intuition: **bugs like this are not implementation bugs, they're architecture bugs.** No amount of writing "better code" inside `settings.js` or `checkout.js` fixes it, because the problem was never inside either one. The fix has to happen at the level of *what's allowed to talk to what* — which is architecture, not implementation.

A useful mental model: think of building a house. - **Implementation** is laying the bricks — the actual code you type. - **Design** is how one room is shaped — a single function or class. - **Architecture** is which rooms have doors to which other rooms — which parts of your app are allowed to talk to which.

Same rooms, same bricks. Architecture is the decision about doors. Get that decision right, and the settings room never had a door into checkout to begin with — the bug becomes structurally impossible, not just unlikely.

Brute Force Solution

The "brute force" fix here isn't a code fix at all — it's the instinctive first move most people make: **patch the symptom.**

The idea: find the exact line in checkout that broke, add a null check or a default value, and move on.

```
# The "just make it stop erroring" patch
total = shared_state.get("cart_total", 0) # defaults to 0 if missing
```

Algorithm: locate the crash, wrap it in a guard, ship it.

Advantages: - Fast. Takes minutes. - Unblocks whatever you were doing.

Disadvantages: - Doesn't explain *why* the value went missing. - The same class of bug can resurface anywhere else that reads `shared_state`, because the actual cause — unrestricted access between unrelated screens — is still there. - Every future "quick guard" like this is a tax you pay forever, and it compounds as the app grows.

Complexity: $O(1)$ time to patch, but unbounded future cost — there's no ceiling on how many times this exact shape of bug can repeat.

Optimal Solution

The optimal fix is to stop treating this as a checkout bug and treat it as a boundary problem: **decide, explicitly, which parts of the app are allowed to depend on which**, and enforce that decision structurally instead of hoping everyone remembers it.

Here's the reasoning, laid out as a table:

Question	What it reveals
What does this piece <i>do</i> ?	Its responsibility — is it settings logic, checkout logic, or shared state?
What does it <i>need</i> to know about?	Its real dependencies — often smaller than what it currently <i>can</i> reach
What's currently allowed to reach it?	The actual door count — often far larger than what's needed

Applying this to the example: settings and checkout both currently have unrestricted access to the same shared state object. Neither *needs* full access to the other's concerns — settings needs to know if a feature flag is on; checkout needs a stable total. Once you separate "settings-owned state" from "checkout-owned state" and give each screen a narrow, explicit interface to read from, a change in

settings can no longer silently shift what checkout sees. The door settings never should have had into checkout gets removed.

Why this scales when the patch doesn't: the fix isn't tied to *this* bug's specific symptom, it's tied to the *shape* of the codebase. Once the boundary exists, this entire category of bug — "unrelated screen breaks" — becomes structurally harder to reproduce, not just fixed once.

Python Code

Here's a minimal illustration of the difference between unrestricted shared state and a bounded interface, in Python terms that map directly to the JS example.

```
# --- Before: no boundary, both screens reach into the same mutable dict ---
shared_state = {}

def settings_add_button(state: dict) -> None:
    # Adds a new read that shifts when a key gets set
    state["feature_flag"] = state.get("cart_total") is not None
    # ^ accidentally depends on checkout's key existing first

def checkout_calculate_total(state: dict) -> float:
    return state["cart_total"] * 1.0 # breaks if key was never set

# --- After: each screen owns its own state, exposed through a narrow interface ---
class CheckoutState:
    def __init__(self, cart_total: float = 0.0):
        self._cart_total = cart_total

    def total(self) -> float:
        return self._cart_total

class SettingsState:
    def __init__(self, checkout: CheckoutState):
        self._checkout = checkout # explicit, narrow dependency

    def feature_flag_enabled(self) -> bool:
        return self._checkout.total() >= 0 # reads through a stable interface
```

Code Walkthrough

- `shared_state = {}` — one mutable dict both "screens" can freely read and write. This is the architectural flaw: no ownership, no boundary.

- `settings_add_button` mutates a key based on the *presence* of another screen's key — this is the "door" that shouldn't exist.
- `checkout_calculate_total` assumes `cart_total` exists, because nothing ever told it that assumption could be violated.
- In the "after" version, `CheckoutState` owns its own data and exposes exactly one method, `total()`. Nothing outside it can set `cart_total` directly.
- `SettingsState` takes a `CheckoutState` as an explicit dependency in its constructor — the relationship is now visible, declared, and limited to one method call instead of open dict access.
- The bug class this prevents: settings can no longer accidentally depend on checkout's *internal* key names or ordering, because it only ever sees `total()`.

Dry Run

Walk through the "before" version with the exact failure from the video:

1. App starts. `shared_state = {}` — empty.
2. User navigates to settings first (order matters, and that's the problem).
`settings_add_button(shared_state)` runs.
3. Inside it: `state.get("cart_total")` — key doesn't exist yet, returns `None`. So `feature_flag` is set to `False`. No crash yet.
4. User navigates to checkout. `checkout_calculate_total(shared_state)` runs.
5. `state["cart_total"]` — this key was never set by anyone in this flow. `KeyError`. Checkout breaks.
6. Nobody edited checkout. The bug was entirely caused by settings reading a key it had no business knowing about, in a shared space with no ownership rules.

Now the "after" version: `SettingsState` can only ever call `checkout.total()`, which always returns a valid float because `CheckoutState` initializes it in its own constructor. There is no path by which settings can put checkout into an invalid state, because settings never has write access to checkout's internals at all.

Complexity Analysis

This isn't a performance problem, so the relevant "complexity" is *structural*, not computational — but it's worth quantifying because it's the actual crux of the lesson.

The number of *possible* connections between `n` files grows as pairs: $n(n-1)/2$.

- At 3 files: 3 possible connections. Trivial to hold in your head.
- At 10 files: 45 possible connections.
- At 50 files: 1,225 possible connections.

This is the ceiling, not a guarantee — but it's the ceiling you're up against as the codebase grows, and it's why "just be careful" stops working long before you'd expect. Space complexity of the *mental model* required to safely change one file is, in the worst case, $O(n)$ — you'd need to hold every other file's assumptions in your head. Boundaries collapse that back down: a change inside a bounded module only requires reasoning about that module's declared interface, not the whole graph.

Alternative Solutions

"Just be more careful" / code review: catches some of these bugs, but doesn't scale — reviewers can't hold a 1,225-edge graph in their head any better than the original author.

Dump shared logic into a `utils.js`: the instinctive fix, and it feels like progress because you've "separated" the logic. It isn't a fix. Now every file imports `utils`, so `utils` silently becomes the single most-connected node in the entire graph — you've hidden the fan-out behind a filename instead of removing it. This is worth calling out explicitly because it's the trap most people fall into on the way to learning boundaries actually work.

Real boundaries (the recommended approach): each module owns its own state and exposes a narrow, explicit interface — as shown above. More upfront thought, but the connection count a human has to reason about per change drops back to a small, fixed number regardless of total app size.

Edge Cases

- **Empty state / first run:** a boundary's constructor should establish valid defaults (as `CheckoutState.__init__` does) so "state doesn't exist yet" can't become "state is silently wrong."
- **Circular dependencies:** if `CheckoutState` also needed to read from `SettingsState`, you'd have a cycle — a sign the boundary was drawn in the wrong place, not that boundaries don't work.
- **Legitimate cross-cutting concerns** (like a logged-in user): these are exactly why "no boundaries anywhere" fails, but also why "everything is its own hard boundary" fails too — some things genuinely need to be shared, deliberately, through an explicit interface rather than an open dict.

- **Very small apps (3 files):** boundaries are actively not worth it here — there's nothing to get lost in yet, and adding structure is pure overhead with no payoff.

Common Mistakes

1. **Assuming the broken file is the buggy file.** Checkout looked broken; checkout wasn't the cause.
2. **Fixing the symptom (null check) instead of the structural cause.** It stops this crash, not this *class* of crash.
3. **Adding a `utils.js` to "reduce coupling."** It usually increases fan-out and just hides it.
4. **Adding heavy structure to a 3-file script "to be safe."** Premature boundaries are pure friction with no payoff at that scale.
5. **Treating architecture as a diagram you draw once.** It's an ongoing rule enforced on every change, not documentation.
6. **Waiting until the app is unmanageable to add boundaries.** The right time is *before* the file count outgrows your memory, not after.

Interview Questions

- "How would you detect that two unrelated modules are secretly coupled?"
- "At what point in a project's growth would you introduce module boundaries, and why not earlier?"
- "Why does moving shared code into a `utils` file not, by itself, reduce coupling?"
- "How do you distinguish an implementation bug from an architecture bug when debugging?"
- "What's the tradeoff of adding boundaries too early versus too late?"

Similar Problems

- **Design a rate limiter / cache with clear module boundaries** — same core skill: deciding what's allowed to depend on what.
- **Refactor a God object** — the direct next step once you've identified unbounded shared state.
- **Design a plugin system** — architecture as explicit interfaces taken to its logical extreme.
- **Microservices vs. monolith tradeoffs** — the same boundary question at a much larger scale.

Key Takeaways

- A change in one file breaking an unrelated file is a sign of missing boundaries, not bad code.

- Architecture is the ongoing decision of what's allowed to depend on what — not a diagram, a rule you enforce continuously.
- Possible connections grow roughly with the square of file count; that's the ceiling you eventually hit.
- Dumping shared code into `utils.js` hides fan-out, it doesn't remove it.
- Boundaries cost upfront thought and are genuinely not worth it at a handful of files — the goal is adding them before file count outgrows your memory, not on day one.

Watch the Video

This lesson is easier to absorb watching the button click and the break happen live. Watch it here: {LINK}

About the Series

This is Lesson 1 of **Fun with Learning Technology**, a series built around one idea: real engineering intuition comes from watching a small, concrete problem break and understanding exactly why — not from memorizing definitions. Future lessons build on this same "one button, two screens" example as the app grows, showing exactly when and how boundaries should evolve with it.

Call To Action

If this changed how you think about a bug you've hit before, the video walks through it visually — worth a watch and a like on YouTube. Subscribe there for the rest of the series, and subscribe to the Substack if you want these lessons as writeups you can reference later. Drop a comment with a time you got burned by exactly this kind of "unrelated" bug — and share this with a teammate who's about to reach for a `utils.js`.

Quick Review

Architecture vs. design vs. implementation — what's the difference?

Architecture = boundaries between parts; design = internal shape of one part; implementation = actual code. Different altitudes, different questions.

Why does 'just start coding' break down as a project grows?

With one file/button, nothing separates the parts, so any change ripples everywhere — the obvious guess stops being obvious past a certain size.

What is a 'boundary' in software architecture?

A deliberate seam where two parts talk only through a defined interface, keeping changes on one side from leaking into the other.

'Boundaries, not blueprints' — what does this mean?

Architecture isn't a fixed upfront diagram; it's the ongoing discipline of keeping parts separated as the system changes.

When are boundaries NOT worth adding yet?

When the system is small (e.g. 'at three files, it's fine') — premature separation adds cost before connections outgrow the code.

What's the 'three questions' test for whether you need architecture?

Ask: what changes together, what changes independently, and where do connections cross — if answers are unclear, that's the signal boundaries are needed.

software-architecture Lesson 1: What Software Architecture Actually Means (architecture vs design vs implementation, why 'just start coding' breaks down past a certain size) — free Python cheat sheet